I have compiled the complete master design module covering all the resort extensions, items, dialogue scripts, and synthesis source code designed throughout this conversation into a clean text document.
You can copy and save this complete overview directly into your expansion archives:
📜 Shoreline Sanctuary: Phase 4 & 5 Complete Resort Expansion Pack
Section 1: Baseline Architecture & Mapped Items
The resort extension introduces placeable, functional tools and environmental decorations crafted at the Sea Glass Artisanal Workshop terminal. Each item serves a tactile audio purpose and a distinct mechanical feedback loop to support top-tier gameplay accessibility.
1. The Beach Umbrella
 * **Blueprint Mechanics:** Crafted using driftwood poles and washed-up canvas sails. Once placed on the sand, it provides interactive shade zones for Snappy the Sea Turtle and local beach crabs.
 * **Audio Profile:** Opening and closing plays a tight, muffled canvas air displacement sweep (120\text{ Hz} \to 45\text{ Hz}). Standing underneath it during a rain shower mutes background high frequencies by 30%, overlaying a cozy, low-mid canvas rain texturescape (*"pitter-patter-thip"*).
 * **Visual Direction:** Storybook style canvas panels stitching together alternating bands of sun-faded teal and warm cream fabric, fixed to a rough driftwood center post.
2. The Picnic Basket
 * **Blueprint Mechanics:** Positioned on a cozy beach blanket to store crafted consumables like Coconut Milk. Serves as the primary anchor for the **Winged Bandit (Seagull Snatch)** event loop.
 * **Audio Profile:** Yields short, bandpass-filtered noise bursts (800\text{ Hz} - 2.8\text{ kHz}) mimicking the creaking friction of woven wicker when lifted, resolving in a tight, resonant dual-tap wooden peg toggle closure. Tapping it empty produces a hollow ring, while filling it adds liquid dampening feedback for low-vision item audits.
 * **Visual Direction:** A classic double-lidded picnic basket woven from dried golden sea oats, lined with a red-and-white checkered cloth pattern and holding seasonal fruit or corked bottles.
3. The Beach Bag
 * **Blueprint Mechanics:** Acts as a permanent mobile inventory extension (+10 slots) enabling the simultaneously expanded transport of sand vials, shells, and coconuts. Discovered buried deep beneath the sand dunes near Snappy the Sea Turtle's primary resting spot.
 * **Audio Profile:** Features a sustained, low-mid frequency friction layer (250\text{ Hz} - 850\text{ Hz}) recreating heavy canvas duck cloth dragging, mixed with dull, low-frequency item shift thumps (130\text{ Hz}).
 * **Visual Direction:** A thick mint-green canvas tote displaying clean vertical cream stripes, fitted with durable, twisted hemp rope handles and stuffed with visible collectible vials.
Section 2: Unified Audio Synthesis Library (Python)
This script uses only native libraries (wave, math, struct, and random) to programmatically output high-quality, 16-bit 44.1 kHz mono .wav files for the standalone items.
```python
import wave
import math
```

```
import struct
import random

SAMPLE_RATE = 44100

def save_wav(filename, data):
    with wave.open(filename, 'wb') as wav_file:
        wav_file.setnchannels(1)
        wav_file.setsampwidth(2)
        wav_file.setframerate(SAMPLE_RATE)
        for sample in data:
            clipped = max(-1.0, min(1.0, sample))
            integer_value = int(clipped * 32767)
            wav_file.writeframesraw(struct.pack('<h', integer_value))
    print(f"✅ Generated: {filename}")

def generate_white_noise(length_samples):
    return [random.uniform(-1.0, 1.0) for _ in range(length_samples)]

def make_umbrella():
    duration = 1.0
    total_samples = int(SAMPLE_RATE * duration)
    output = [0.0] * total_samples
    noise = generate_white_noise(total_samples)
    for t in range(int(SAMPLE_RATE * 0.4)):
        progress = t / (SAMPLE_RATE * 0.4)
        window_size = int(5 + 180 * progress)
        sum_noise = 0.0
        for w in range(-window_size // 2, window_size // 2):
            idx = max(0, min(total_samples - 1, t + w))
            sum_noise += noise[idx]
        avg_noise = sum_noise / window_size
        amp = math.sin(progress * math.pi) * 0.4
        output[t] += avg_noise * amp
    click_start = int(SAMPLE_RATE * 0.22)
    for t in range(int(SAMPLE_RATE * 0.15)):
        time_sec = t / SAMPLE_RATE
        wave1 = math.sin(2 * math.pi * 1100 * time_sec) * 0.4
        wave2 = math.sin(2 * math.pi * 900 * time_sec) * 0.2
        decay = math.exp(-180 * time_sec)
        target_idx = click_start + t
        if target_idx < total_samples:
            output[target_idx] += (wave1 + wave2) * decay
    return output
```

```
def make_picnic():
    duration = 0.8
    total_samples = int(SAMPLE_RATE * duration)
    output = [0.0] * total_samples
    noise = generate_white_noise(total_samples)
    for i in range(15):
        rustle_time = random.uniform(0.0, 0.3)
        rustle_sample = int(rustle_time * SAMPLE_RATE)
        rustle_len = int(random.uniform(0.02, 0.06) * SAMPLE_RATE)
        for t in range(rustle_len):
            target_idx = rustle_sample + t
            if target_idx >= total_samples:
                break
            p = t / rustle_len
            envelope = math.sin(p * math.pi) * 0.05
            output[target_idx] += noise[target_idx] * envelope
    latch_start = int(SAMPLE_RATE * 0.35)
    for t in range(int(SAMPLE_RATE * 0.2)):
        time_sec = t / SAMPLE_RATE
        wave_peg = math.sin(2 * math.pi * 320 * time_sec) * 0.5
        decay = math.exp(-120 * time_sec)
        target_idx = latch_start + t
        if target_idx < total_samples:
            output[target_idx] += wave_peg * decay
    return output

def make_beach_bag():
    duration = 1.2
    total_samples = int(SAMPLE_RATE * duration)
    output = [0.0] * total_samples
    noise = generate_white_noise(total_samples)
    for t in range(int(SAMPLE_RATE * 0.8)):
        progress = t / (SAMPLE_RATE * 0.8)
        window_size = 90
        sum_noise = 0.0
        for w in range(-window_size // 2, window_size // 2):
            idx = max(0, min(total_samples - 1, t + w))
            sum_noise += noise[idx]
        filtered_noise = sum_noise / window_size
        amp = math.sin(progress * math.pi) * 0.25
        output[t] += filtered_noise * amp
    thud_times = [0.2, 0.45]
    for thud_start_sec in thud_times:
```

```
        thud_start = int(SAMPLE_RATE * thud_start_sec)
        for t in range(int(SAMPLE_RATE * 0.3)):
            time_sec = t / SAMPLE_RATE
            thump = math.sin(2 * math.pi * 130 * time_sec) * 0.3
            decay = math.exp(-65 * time_sec)
            target_idx = thud_start + t
            if target_idx < total_samples:
                output[target_idx] += thump * decay
    return output

if __name__ == "__main__":
    save_wav("umbrella_whoof.wav", make_umbrella())
    save_wav("picnic_latch.wav", make_picnic())
    save_wav("bag_rustle.wav", make_beach_bag())

```
Section 3: Winged Bandit (Seagull) Dialogue Scripts
Seagull encounters feature expressive bird noises translated to comical thoughts, structured natively for screen readers.
Encounter 1: The Picnic Basket Swoop
 * **Context:** A seagull lands near an unlatched basket holding food items.
 * **Dialogue:** * **Cheeky Seagull:** "SQUAWK! *(Translation: That is a very nice-looking snack you have there. It would be a shame if... someone raided it.)*"
 * **Options:** [Trade a Coconut Milk] / [Wave hand to shoo it away]
 * **Outcome - Trade:**
   * **Cheeky Seagull:** "Heh... *Kerr-r-r!* *(Translation: Excellent doing business with you, human. A fair swap!)*"
   * **System:** *The seagull drops its treasure into your basket with a tiny plastic clack and happily rolls away with your bottle of Coconut Milk!* (Receives: [Lost Key], [Shiny Soda Tab], or [Rare Sea Glass])
 * **Outcome - Shoo:**
   * **Cheeky Seagull:** "HRAAAK! *(Translation: Hmph! Keep your fancy milk. I'll just find another blanket...)*"
   * **System:** *With a disappointed ruffle of feathers, the seagull takes off back into the sea breeze.*
Encounter 2: Snappy's Defensive Response
 * **Context:** Triggers if the player sets up the Picnic Basket next to Snappy the Sea Turtle. Snappy initiates an extremely slow defense mechanism.
 * **Dialogue:**
   * **Cheeky Seagull:** "Mine? *Kerrr...* *(Translation: Target spotted. Left lid unlatched. Proceeding with the heist.)*"
   * **Snappy the Sea Turtle:** "Hsssssssssss... *(Translation: Hey... you... drop... that... right... now... please...)*"
   * **Cheeky Seagull:** "SQUAWK! *(Translation: Yikes! The rock has a mouth!)*"

 * **Branching Success (Cheering Snappy / Shooing Bird):**
   * **System:** *The startled seagull trips over a sand dune pocket, drops the item, and flees in a flurry of feathers.*
   * **Snappy the Sea Turtle:** "*Coo-click.* *(Translation: Crisis averted. Nobody mess with my favorite human's basket. Back to my nap.)*" (Receives: [Snappy Happiness +15] and bonus [Polished Teal Sea Glass])
 * **Branching Delay (Allowing the Swap):**
   * **Snappy the Sea Turtle:** "...ssssss! *Chomp.* ...Oh. I missed him. *Sigh.* Those birds have no respect for golden-hour relaxation tempos." (Receives: [Lost Key] or [Shiny Soda Tab])
Section 4: Message in a Bottle Quest Lines
Letters wash ashore in frosted bottles. Opening a bottle triggers a cork vacuum pop (*"shhh-pop"*), unfolding parchment sounds, and semantic reader assignments.
✉️ Letter 1: The Picnic Basket Panic (Sender: Oliver, the Distapped Traveler)
 * **Body:** *"Hello friend! If you are reading this, I hope your blanket is firmly anchored. I set up a beautiful spot near the dunes yesterday afternoon, opened my woven wicker basket, and poured a cold glass of Coconut Milk. The view was magnificent! But then... it happened. A shadow fell, a thunderous flap of wings echoed, and a chubby seagull dive-bombed right into my snacks! In the panic, I dropped my favorite journal somewhere in the sand. Could you help me find it? I hear those birds love trading shiny things if you leave a treat out for them."*
 * **Objective:** Place a Picnic Basket on the shore and trade a Coconut Milk to a seagull to retrieve Oliver's journal.
 * **Rewards:** [Rare Blueprint: Vintage Beach Blanket] & [5x Polished Teal Sea Glass]
✉️ Letter 2: Snappy’s Sunscreen Alternative (Sender: Marina, the Marine Biologist)
 * **Body:** *"Greetings from across the bay! I’ve been monitoring the local shorelines, and I noticed that Snappy the Sea Turtle has been spending a lot of time resting up on the hot sand dunes lately. While he loves the afternoon heat, the mid-day sun can get a bit too intense for his shell! We need a zero-stress way to keep him cool while he naps. I've sent over a design to piece together a shade shelter. If you can gather enough driftwood and washed-up canvas from the shore, we can set up a personal canopy right over his favorite resting spot!"*
 * **Objective:** Craft a Beach Umbrella at the station and place it next to Snappy.
 * **Rewards:** [Snappy’s Blessing (Increases local crab happiness)] & [1x Sparkling Golden Pearl]
✉️ Letter 3: The Lost Luggage Mystery (Sender: Captain Vance, of the Stray Rowboat)
 * **Body:** *"To whoever finds this message! My cargo ship caught a rogue wave just off the Hidden Cove last week. While the ship survived, a heavy gust of wind caught our supply deck and sent several of our newly tailored inventory expansion bags tumbling right into the surf. They are incredibly tough, made of heavy canvas duck cloth and twisted hemp rope, so they won't get ruined by the salt—but the tide has likely buried them deep under the coastal dunes by now. Keep an eye out for a mint-green strap peeking through the sand oats!"*
 * **Objective:** Locate and unearth the buried Beach Bag from the Lighthouse Lookout dunes.
 * **Rewards:** [Permanent Mobile Inventory Expansion (+10 Slots)] & [12x Shiny Soda Tabs]
Section 5: Advanced Workshop Recipes & Raft Protection Loops
Craft Card 1: Inflatable Rubber Raft

 * **Ingredients Required:** 4/4 Floating Beach Balls, 2/2 Recycled Rubber Strips, 1/1 Weathered Driftwood Oar.
 * **UI Assembly Method:** Press & hold the active golden button for 2.5 seconds to fill the progress gauge.
 * ** Sensory Audio:** Air-pump hand inflation cycles (*"pfft-shhh"*) give way to heavy rope knot friction squeaks (*"creak-sqwuk"*), concluding in a clear sea glass marble chime (*"tink-ting!"*).
 * **Mechanical Unlock:** Permanently links the workshop terminal to the **Shifting Sandbars** open-water exploration zone.
🛡️ Sandbar Defense Loop: The Floating Trampoline
Triggers randomly when the raft is left un-anchored near the sandbars, attracting birds who view it as a bouncing surface.
 * **Options:** [Toss a Kelp Chip snack] / [Gently splash water toward the raft]
 * **Outcome - Kelp Chips:** Birds fly to a flat rock to eat, ignoring the raft. (Grants: [Raft Integrity Maintained] & [Seagull Friendship +10])
 * **Outcome - Splash:** Water spray safely startles the seagulls away onto adjacent sandbars.
 * **Outcome - Ignore (Deflation):** A bird pulls the string on the air release valve. The raft sags comically like a melting scoop of mint ice cream into a flat state (*"pffFSSSSSSSSssssss..."*). It automatically returns to the workshop terminal dock, requiring no materials to re-inflate—simply hold down the pump gauge for 2 seconds.
Section 6: Meditative Crafting Station Expansions
Craft Card 2: Sand Art Layering
 * **Biomes For Raw Sand Gathering:** Pastel Pink Sand (Dune Sea-Roses), Deep Ocean Teal Sand (Tide line edge), Warm Apricot Sand (Lighthouse Lookout dry expanse).
 * **The Recipe Log:** * *Sunset Shoreline:* 2x Apricot Sand, 2x Pink Sand, 1x Frosted Bottle. Alternating flat blocks. Casts a permanent golden-hour environment color shader overlay around local blankets.
   * *Sub-Aquatic Sandbar:* 3x Teal Sand, 1x Apricot Sand, 1x Barnacle-Encrusted Bottle. Generates a small ripple wave layer in the glass. Attracts following sandbar crabs.
   * *Legendary Tide-Pool Glimmer:* 2x Pink Sand, 2x Teal Sand, 1x Sparkling Golden Pearl, 1x Turquoise Sealed Bottle. Crushing the pearl at the mortar station mixes sparkling dust into both sand stocks. Layered in fine, wavy lines. Unlocks a musical chime ambient loop.
 * **Pouring SFX:** Sifting jars triggers granular maraca noises (*"shhh"*), pitch-compressed relative to sand density (Teal is wet and low; Apricot is dry and crisp). Layer pouring emits cascading waterfalls (*"ssss"*), rising in tone pitch as the air pocket inside the glass neck decreases, secured with a wood cork thud (*"plop-clack"*). A soft guitar harmonic scale plucks sequentially on every layer completion to assist low-vision layout tracking.
Craft Card 3: The Coastal Salt Lamp
 * **Ingredients Required:** 3/3 Raw Pink Salt Crystals (Hidden Cove rock hollows), 1/1 Polished Driftwood Base (Flat ocean-tumbled shoreline oak), 1/1 Glowing Firefly Jar (Sea-rose bushes at twilight).
 * **Visual Silhouette:** A jagged, sharp, geometrically faceted block of pink salt crystal bonded onto a dark, perfectly level circular driftwood platform, emitting a soothing peach-orange glow.

 * **Assembly SFX:** Hardwood sanding friction (*"shhh-rub"*) paired with a heavy mineral placement chink (*"clink-thud"*), resolving in a low 5 Hz oscillating living firefly jar background pulse.
Craft Card 4: Woven Sun Hat
 * **Ingredients Required:** 6/6 Dried Sea Oats (Lighthouse beach grasses), 2/2 Pastel Pink Sea Roses (Dune wild bushes), 1/1 Shiny Soda Tab Buckle (Seagull trade salvage).
 * **UI Status & Benefit:** Wide-brimmed golden straw cosmetic accessory. Equipping the item expands the evening horizon window by **5 real-world minutes** before night sets in, elongating beachcombing runs.
 * **Assembly SFX:** Overlapping high-passed dried straw friction crunching layers (*"hat_straw_weave.wav"*), soft leaf band placement brushes (*"hat_band_wrap.wav"*), and a clear, thin light-metal buckle secure snap (*"hat_buckle_clink.wav"*).
Section 7: Standalone Salt Lamp & Hat Audio Synthesis (Python)
This dedicated module outputs the asset files (lamp_base_sanding.wav, lamp_salt_placement.wav, lamp_firefly_hum.wav, hat_straw_weave.wav, hat_band_wrap.wav, hat_buckle_clink.wav) directly to your build index using native arithmetic calculations.
```python
import wave
import math
import struct
import random

SAMPLE_RATE = 44100

def export_wav(filename, audio_signals):
    with wave.open(filename, 'wb') as wav_output:
        wav_output.setnchannels(1)
        wav_output.setsampwidth(2)
        wav_output.setframerate(SAMPLE_RATE)
        for sample in audio_signals:
            safe_sample = max(-1.0, min(1.0, sample))
            int_representation = int(safe_sample * 32767)
            wav_output.writeframesraw(struct.pack('<h', int_representation))
    print(f"--> Extracted: {filename}")

def build_raw_noise(sample_length):
    return [random.uniform(-1.0, 1.0) for _ in range(sample_length)]

def generate_base_sanding():
    duration = 1.2
    total_samples = int(SAMPLE_RATE * duration)
    output = [0.0] * total_samples
    white_noise = build_raw_noise(total_samples)
    stroke_intervals = [(0.05, 0.5), (0.6, 1.1)]
```

```
    for start_sec, end_sec in stroke_intervals:
        start_idx = int(start_sec * SAMPLE_RATE)
        end_idx = int(end_sec * SAMPLE_RATE)
        span = end_idx - start_idx
        muffle_window = 45
        for t in range(span):
            curr_idx = start_idx + t
            sum_noise = 0.0
            for w in range(-muffle_window // 2, muffle_window // 2):
                look_idx = max(0, min(total_samples - 1, curr_idx + w))
                sum_noise += white_noise[look_idx]
            filtered_grain = sum_noise / muffle_window
            progress = t / span
            amplitude_envelope = math.sin(progress * math.pi) * 0.18
            output[curr_idx] += filtered_grain * amplitude_envelope
    return output

def generate_crystal_placement():
    duration = 0.6
    total_samples = int(SAMPLE_RATE * duration)
    output = [0.0] * total_samples
    for t in range(int(SAMPLE_RATE * 0.15)):
        time_sec = t / SAMPLE_RATE
        crystal_ring = math.sin(2 * math.pi * 1150 * time_sec) * 0.25
        stone_node = math.sin(2 * math.pi * 820 * time_sec) * 0.15
        decay_envelope = math.exp(-180 * time_sec)
        output[t] += (crystal_ring + stone_node) * decay_envelope
    thud_start_offset = int(SAMPLE_RATE * 0.015)
    for t in range(int(SAMPLE_RATE * 0.25)):
        time_sec = t / SAMPLE_RATE
        base_resonance = math.sin(2 * math.pi * 140 * time_sec) * 0.4
        damp_envelope = math.exp(-55 * time_sec)
        target_idx = thud_start_offset + t
        if target_idx < total_samples:
            output[target_idx] += base_resonance * damp_envelope
    return output

def generate_activation_hum():
    duration = 2.0
    total_samples = int(SAMPLE_RATE * duration)
    output = [0.0] * total_samples
    for t in range(total_samples):
        time_sec = t / SAMPLE_RATE
        progress = t / total_samples
```

```
        warm_glow_carrier = math.sin(2 * math.pi * 95 * time_sec) * 0.3
        bug_flutter_layer = math.sin(2 * math.pi * 190 * time_sec) * 0.08
        jar_wobble_lfo = 1.0 + (0.35 * math.sin(2 * math.pi * 5.0 * time_sec))
        if progress < 0.2:
            fade = progress / 0.2
        elif progress > 0.8:
            fade = (1.0 - progress) / 0.2
        else:
            fade = 1.0
        output[t] = (warm_glow_carrier + bug_flutter_layer) * jar_wobble_lfo * fade * 0.35
    return output

def generate_straw_weaving():
    duration = 1.4
    total_samples = int(SAMPLE_RATE * duration)
    output = [0.0] * total_samples
    white_noise = build_raw_noise(total_samples)
    rustle_times = [0.1, 0.3, 0.55, 0.8, 1.05]
    rustle_span = int(SAMPLE_RATE * 0.12)
    for start_sec in rustle_times:
        start_idx = int(start_sec * SAMPLE_RATE)
        window = 8
        for t in range(rustle_span):
            curr_idx = start_idx + t
            if curr_idx < total_samples:
                high_frequency_grain = white_noise[curr_idx] - (white_noise[max(0, curr_idx -
window)] / window)
                progress = t / rustle_span
                amp_envelope = math.sin(progress * math.pi) * 0.15
                output[curr_idx] += high_frequency_grain * amp_envelope
    return output

def generate_band_placement():
    duration = 0.5
    total_samples = int(SAMPLE_RATE * duration)
    output = [0.0] * total_samples
    white_noise = build_raw_noise(total_samples)
    muffle_window = 85
    for t in range(total_samples):
        progress = t / total_samples
        running_sum = 0.0
        for w in range(-muffle_window // 2, muffle_window // 2):
            look_idx = max(0, min(total_samples - 1, t + w))
            running_sum += white_noise[look_idx]
```

```
        soft_friction = running_sum / muffle_window
        amp = math.sin(progress * math.pi) * 0.2
        output[t] = soft_friction * amp
    return output

def generate_buckle_clink():
    duration = 0.3
    total_samples = int(SAMPLE_RATE * duration)
    output = [0.0] * total_samples
    for t in range(int(SAMPLE_RATE * 0.1)):
        time_sec = t / SAMPLE_RATE
        freq_1 = math.sin(2 * math.pi * 1450 * time_sec) * 0.3
        freq_2 = math.sin(2 * math.pi * 1820 * time_sec) * 0.15
        decay = math.exp(-140 * time_sec)
        output[t] = (freq_1 + freq_2) * decay
    return output

if __name__ == "__main__":
    export_wav("lamp_base_sanding.wav", generate_base_sanding())
    export_wav("lamp_salt_placement.wav", generate_crystal_placement())
    export_wav("lamp_firefly_hum.wav", generate_activation_hum())
    export_wav("hat_straw_weave.wav", generate_straw_weaving())
    export_wav("hat_band_wrap.wav", generate_band_placement())
    export_wav("hat_buckle_clink.wav", generate_buckle_clink())

```
```
```